

Práctica 8 — Clase 04

Diseño de capa Redis para FlowOps: caché, estado rápido, sesiones y colas

Materia: Ingeniería de Datos II

Clase: 04 — Bases de Grafos y Bases Clave/Valor

Vinculación con el TPO: Esta práctica permite incorporar Redis como componente clave-valor dentro de la arquitectura políglota de FlowOps.

1. Contexto

FlowOps necesita ejecutar procesos operativos de manera ágil. Algunas consultas y operaciones no requieren recorrer todo el historial ni consultar siempre la base documental o columnar.

Ejemplos:

- Consultar rápidamente el estado actual de una instancia.
- Mantener una sesión de usuario activa.
- Guardar temporalmente la configuración de un tenant.
- Llevar contadores de tareas pendientes.
- Implementar límites de uso por tenant.
- Mantener una cola simple de trabajos.
- Gestionar notificaciones pendientes.
- Cachear resultados frecuentes.

Redis puede aportar velocidad, TTL, estructuras de datos en memoria y patrones operativos útiles, siempre que se use para los problemas correctos.

2. Contenidos utilizados

Esta práctica se basa en los contenidos de la clase sobre Redis:

- Modelo clave-valor.
- Base de datos en memoria.
- Strings.

- Hashes.
- Lists.
- Sets.
- Sorted Sets.
- Streams.
- TTL y expiración.
- Caché.
- Sesiones.
- Contadores.
- Colas.
- Rate limiting.
- Persistencia configurable.
- Políticas de memoria.
- Casos donde Redis es adecuado y casos donde no lo es.

No se exige todavía:

- Conectar Redis desde una aplicación real.
- Implementar workers.
- Desarrollar un backend funcional.
- Usar Redis Cluster.
- Hacer benchmarking.
- Usar Lua scripts.
- Configurar monitoreo avanzado.

3. Objetivo de la práctica

El grupo deberá diseñar una **capa Redis conceptual para FlowOps**, indicando qué datos guardarían en Redis, con qué estructura, bajo qué clave, durante cuánto tiempo, con qué operación principal y qué riesgo implica usar memoria para ese caso.

4. Parte A — Identificación de candidatos para Redis

Analicen los siguientes datos de FlowOps y decidan si son buenos candidatos para Redis.

Dato o función	¿Usar Redis?	Motivo	Riesgo
Estado actual de una instancia activa	Sí / No / Parcial		

Dato o función	¿Usar Redis?	Motivo	Riesgo
Historial completo de eventos	Sí / No / Parcial		
Definición publicada de un proceso	Sí / No / Parcial		
Sesión de usuario autenticado	Sí / No / Parcial		
Tareas pendientes por rol	Sí / No / Parcial		
Contador de instancias iniciadas por día	Sí / No / Parcial		
Rate limit por tenant	Sí / No / Parcial		
Cola de notificaciones pendientes	Sí / No / Parcial		
Métricas agregadas temporales	Sí / No / Parcial		
Datos maestros del tenant	Sí / No / Parcial		

Luego respondan:

1. ¿Qué datos deberían estar siempre en una base persistente además de Redis?
2. ¿Qué datos podrían perderse sin comprometer la auditoría?
3. ¿Qué datos deberían tener TTL?
4. ¿Qué datos no deberían guardarse en Redis?

5. Parte B — Diseño de claves

Diseñen una convención de nombres de claves para FlowOps.

Ejemplos:

tenant:{tenant_id}:config

session:{session_id}

instance:{tenant_id}:{instance_id}:state

tasks:{tenant_id}:{role}:pending

metrics:{tenant_id}:{yyyyMMdd}:instances_started

rate_limit:{tenant_id}:{window}

queue:{tenant_id}:notifications

Completen la tabla:

Uso	Clave propuesta	Ejemplo concreto	Motivo
Configuración de tenant			
Sesión de usuario			
Estado actual de instancia			
Tareas pendientes por rol			
Contador diario			
Rate limit por tenant			
Cola de notificaciones			
Cache de definición de proceso			

6. Parte C — Selección de estructuras Redis

Seleccionen qué estructura Redis usarían para cada caso.

Estructuras posibles:

- `String`.
- `Hash`.
- `List`.
- `Set`.
- `Sorted Set`.
- `Stream`.

Caso	Estructura Redis	Justificación	Operaciones principales
Sesión de usuario			
Estado actual de instancia			
Tareas pendientes por rol			
Usuarios conectados por tenant			
Ranking de procesos más ejecutados			
Cola de notificaciones			
Eventos recientes para monitoreo			
Rate limit por tenant			

Ejemplo:

Caso: estado actual de instancia

Estructura: Hash

Clave: instance:acme:inst_001:state

Campos: status, current_node, updated_at, assigned_role

Operaciones: HSET, HGETALL, EXPIRE

7. Parte D — TTL y expiración

Definan qué claves deberían expirar y en cuánto tiempo.

Clave o tipo de dato	¿Tiene TTL?	TTL propuesto	Justificación
Sesión de usuario	Sí / No		
Cache de proceso publicado	Sí / No		
Estado de instancia activa	Sí / No		
Instancia finalizada	Sí / No		
Rate limit	Sí / No		
Contador diario	Sí / No		
Cola de notificaciones	Sí / No		
Métricas temporales	Sí / No		

Preguntas guía:

1. ¿Qué ocurre si una clave expira demasiado rápido?
2. ¿Qué ocurre si nunca expira?
3. ¿Qué datos requieren invalidación manual?
4. ¿Qué datos pueden regenerarse desde MongoDB o Cassandra?
5. ¿Qué datos no deberían depender exclusivamente de Redis?

8. Parte E — Patrones de uso

8.1 Cache-aside para definición de proceso

Diseñen el flujo conceptual para obtener la definición publicada de un proceso.

Completen:

1. La API recibe una solicitud para iniciar una instancia.
2. Busca en Redis la clave: _____
3. Si existe:
 - Usa la definición cacheada.
4. Si no existe:
 - Consulta la base persistente.
 - Guarda en Redis con TTL de _____.
 - Usa la definición recuperada.
5. Si el proceso cambia:
 - Se debe _____.

Indiquen qué riesgos existen si se usa una definición desactualizada.

8.2 Sesiones de usuario

Diseñen la estructura de una sesión:

```
{  
  
  "user_id": "usuario_123",  
  
  "tenant_id": "tenant_acme",  
  
  "role": "compras",  
  
  "created_at": "2026-06-09T10:00:00Z",  
  
  "last_activity": "2026-06-09T10:25:00Z"  
}
```

Completen:

Decisión	Respuesta
Clave Redis	
Estructura	
TTL	
Operación para crear	
Operación para renovar	
Operación para cerrar sesión	
Riesgo principal	

8.3 Tareas pendientes por rol

Diseñen cómo guardarían las tareas pendientes para un rol.

Ejemplo:

tasks:tenant_acme:compras:pending

Preguntas:

1. ¿Usarían List, Set o Sorted Set?
2. ¿Necesitan conservar orden?
3. ¿Necesitan prioridad?
4. ¿Una tarea puede estar duplicada?
5. ¿Qué pasa si Redis se reinicia?
6. ¿Dónde queda la versión persistente de la tarea?

8.4 Rate limit por tenant

Diseñen un rate limit simple para evitar que un tenant abuse de la API.

Ejemplo:

rate_limit:tenant_acme:20260609T1030

Completen:

Parámetro	Valor propuesto
Ventana de tiempo	
Máximo de solicitudes	
Estructura Redis	
Operación principal	
TTL	
Respuesta si supera el límite	

9. Parte F — Comandos Redis esperados

Propongan comandos para al menos cinco casos.

Caso 1 — Crear sesión

SET session:sess_123 '{"user_id":"u1","tenant_id":"acme","role":"compras"}' EX 3600

Caso 2 — Guardar estado actual

HSET instance:acme:inst_001:state status "waiting_task" current_node "revision_compras" updated_at "2026-06-09T10:30:00Z"

EXPIRE instance:acme:inst_001:state 86400

Caso 3 — Incrementar contador diario

INCR metrics:acme:20260609:instances_started

EXPIRE metrics:acme:20260609:instances_started 2592000

Ahora completen:

Caso	Comando Redis propuesto	Explicación
Crear sesión		
Renovar sesión		
Guardar estado actual		
Agregar tarea pendiente		
Completar tarea pendiente		
Incrementar contador diario		
Aplicar rate limit		
Encolar notificación		

10. Parte G — Riesgos de Redis

Analicen los siguientes riesgos:

Riesgo 1 — Pérdida de datos en memoria

Responder:

1. ¿Qué datos no deberían depender exclusivamente de Redis?
 2. ¿Qué datos pueden reconstruirse?
 3. ¿Qué persistencia configurarían conceptualmente: RDB, AOF o ninguna?
 4. ¿Qué impacto tendría perder sesiones activas?
-

Riesgo 2 — Datos desactualizados en caché

Responder:

1. ¿Qué pasa si cambia una definición de proceso y Redis mantiene una versión vieja?
 2. ¿Qué estrategia usarían: TTL corto, invalidación manual o versionado?
 3. ¿Cómo lo relacionan con consistencia eventual?
-

Riesgo 3 — Uso excesivo de memoria

Responder:

1. ¿Qué claves podrían crecer sin control?
2. ¿Qué TTL o límite aplicarían?
3. ¿Qué política de eviction sería razonable?
4. ¿Qué datos nunca deberían ser expulsados sin respaldo persistente?

11. Parte H — Integración con la arquitectura polígloa

Actualicen la matriz de persistencia del TPO incorporando Redis.

Subsistema de FlowOps	Base principal	¿Redis interviene?	Uso de Redis
Definiciones de procesos	MongoDB / otra	Sí / No	
Estado actual de instancia	MongoDB / otra	Sí / No	
Eventos de auditoría	Cassandra / otra	Sí / No	
Tareas humanas	MongoDB / otra	Sí / No	
Sesiones	Redis / otra	Sí / No	
Métricas temporales	Redis / otra	Sí / No	
Notificaciones	Redis / otra	Sí / No	
Validación de flujo	Neo4j / otra	Sí / No	

12. Entregable

Cada grupo deberá entregar un documento breve, de 3 a 5 páginas, con:

1. Identificación de candidatos para Redis.
2. Convención de claves.
3. Selección de estructuras Redis.
4. Definición de TTL.
5. Diseño de cuatro patrones: caché, sesiones, tareas y rate limit.
6. Comandos Redis propuestos.
7. Análisis de riesgos.
8. Matriz de integración con el TPO.

13. Puesta en común

Cada grupo tendrá 4 minutos para explicar:

¿Cuál es el uso más valioso de Redis dentro de su FlowOps y cuál sería el riesgo si lo usan incorrectamente?

La exposición debe incluir:

- Una clave Redis concreta.
- Una estructura Redis concreta.
- Un TTL o política de expiración.
- Una advertencia sobre persistencia o consistencia.

14. Criterios de evaluación

Criterio	Peso
Selección correcta de usos para Redis	20%
Diseño de claves claro y consistente	20%
Selección adecuada de estructuras Redis	25%
Uso razonado de TTL y expiración	15%
Análisis de riesgos	15%
Claridad de la puesta en común	5%

15. Restricciones

No se evaluará:

- Backend funcional.
- Workers reales.
- Redis Cluster.
- Benchmarking.
- Scripts Lua.
- Monitoreo avanzado.

Sí se evaluará:

- Comprensión del modelo clave-valor.
- Elección correcta de estructuras.
- Diseño de claves.
- TTL.
- Riesgos de usar Redis como almacenamiento principal.
- Aplicación al TPO FlowOps.